

DRL TASK 2 (WS-2025)

Maneet Singh
OTH Amberg-Weiden

February 9, 2026

1 Challenge: Autonomous Navigation in the "Parcour" Environment

The primary objective is to develop an autonomous agent capable of navigating a complex **Parcour** circuit consisting of 34 sequential gates. The environment presents significant challenges:

- **Complex Geometry:** High-curvature turns and 180-degree hairpins require precise momentum management and trajectory planning.
- **Stochastic Dynamics:** The environment injects Gaussian noise into the acceleration commands. With *ADD_ACC_NOISE* enabled, noise is calculated as:

$$\sigma_{trans} = \frac{a_{limit}}{5}, \quad \sigma_{rot} = \frac{\alpha_{limit}}{5} \quad (1)$$

- **Velocity-Dependent Constraints:** Rotational velocity is constrained by the current translational speed to simulate realistic vehicle dynamics:

$$v_{rot,limit} = V_{rot,max} \left(1 - (1 - k) \frac{|v_{trans}|}{V_{trans,max}} \right) \quad (2)$$

where $k = 0.3$. This coupling makes high-speed cornering impossible without braking.

1.1 The RL Solution: PPO Algorithm

To solve this continuous control problem under uncertainty, I implemented the **Proximal Policy Optimization (PPO)** algorithm. PPO was chosen for its stability in continuous action spaces and its ability to learn robust policies in stochastic environments where manual control is difficult.

2 State and Action Representation

The reinforcement learning agent operates within a continuous state and action space defined as follows:

2.1 State Space

The observation vector $s \in \mathbb{R}^5$ describes the kinematic state of the robot at each time step t :

$$s = [x, y, \theta, v_{trans}, v_{rot}]^T \quad (3)$$

Where:

- (x, y) : Cartesian coordinates of the robot center.
- θ : Current heading (orientation) in radians.
- v_{trans} : Translational velocity in m/s .
- v_{rot} : Rotational velocity in rad/s .

2.2 Action Space

The action vector $a \in \mathbb{R}^2$ consists of continuous acceleration commands:

$$a = [a_{trans}, a_{rot}]^T \quad (4)$$

The actions are clipped by the environment to the following physical limits:

- $a_{trans} \in [-0.5, 0.5] m/s^2$
- $a_{rot} \in [-0.2, 0.2] rad/s^2$

The transition to the next state s_{t+1} is calculated using the Euler integration of these accelerations over $DT = 0.2s$, including the stochastic noise component.

3 Environment Wrapping and State Augmentation

To facilitate training with the PPO algorithm, the raw environment was wrapped in a **Gymnasium** interface. This process involved transforming the global coordinate system into a relative, local coordinate system to improve the agent’s ability to generalize.

3.1 State Augmentation

The augmented observation vector $s' \in \mathbb{R}^5$ is defined relative to the target gate:

$$s' = [dist, \sin(\phi), \cos(\phi), v, \omega]^T \quad (5)$$

where $dist$ is the Euclidean distance to the gate center and ϕ is the angular error. By using trigonometric components ($\sin, \cos$), I eliminate the mathematical discontinuity at $\pm\pi$, providing a smooth gradient for the neural network.

3.2 Reward Shaping

The reward function was redesigned from a sparse signal to a dense potential-based signal to guide exploration. The total reward r_t is calculated as:

$$r_t = r_{gate} + w_{dist}(d_{t-1} - d_t) + w_{align}(v \cdot \cos \phi) - w_{time} - \mathcal{P} \quad (6)$$

Where:

- r_{gate} : Sparse reward for crossing a gate.
- $w_{dist}(d_{t-1} - d_t)$: Reward for reducing distance to the target.
- w_{align} : Incentive for facing the gate while maintaining velocity.
- $\mathcal{P}$: Penalties for excessive angular velocity and idling.

Truncation logic was also implemented to penalize "stagnation" (no progress for 200 steps) or "wandering" (distance $> 15m$), ensuring the agent stays focused on the track boundaries.

4 PPO Algorithm and Implementation

The agent’s policy is optimized using Proximal Policy Optimization (PPO). The architecture consists of an Actor-Critic framework where the policy $\pi_\theta(a|s)$ and value function $V_\phi(s)$ are parameterized by separate neural networks with shared tanh activations. [1]

4.1 PPO Objectives

To ensure stable updates, the policy loss is clipped using a surrogate objective:

$$L^{CLIP} = \mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \quad (7)$$

The total loss function also includes a value function squared error ($c_1 = 0.5$) and an entropy bonus to encourage exploration.

5 Results

The performance of the PPO agent was benchmarked using the average of 10 independent seeds to ensure statistical significance. The results focus on sample efficiency (computational cost) and policy optimality (speed).

5.1 Key Performance Indicators

The three critical metrics required for this evaluation are summarized in Table 1.

Table 1: Aggregated Performance Metrics (10 Seeds)

Metric	Value (Mean $\pm$ SD)
Calls to <code>_calc_next_state</code> for first solve	[6969.8] $\pm$ [483.1]
Minimum episode time steps to solve Parcour	[386] steps
Calls to <code>_calc_next_state</code> to reach best solve	[83375.1] calls

6 Custom Archery Reinforcement Learning Environment

A custom OpenAI Gym-compatible environment (**ArcheryEnv**) was developed to train an agent to solve a projectile motion task. The objective is to launch an arrow from a fixed starting position to hit a target placed at a randomized location, accounting for gravity and variable wind conditions.

6.1 Environment Dynamics

The environment simulates 2D projectile motion under gravity ($g = 9.81 \text{ m/s}^2$) and wind resistance. The simulation uses a "one-shot" dynamics model where a single action determines the entire trajectory of the arrow. The physics update at each time step Δt follows the Euler integration method:

$$\begin{aligned} x_{t+1} &= x_t + v_{x,t} \Delta t \\ y_{t+1} &= y_t + v_{y,t} \Delta t \\ v_{x,t+1} &= v_{x,t} + \text{wind} \cdot \Delta t \\ v_{y,t+1} &= v_{y,t} - g \cdot \Delta t \end{aligned}$$

Collision detection is performed using line-segment intersection to ensure fast-moving projectiles do not tunnel through the target.

6.2 Observation Space (State)

The observation space is a continuous 5-dimensional vector representing the relative geometry of the task and environmental factors. The state S_t is defined as:

$$S_t = [T_x, T_y, d, \theta_{target}, w]$$

- T_x, T_y : The absolute coordinates of the target center (meters).
- d : Euclidean distance from the launch point to the target center.
- θ_{target} : The angle (radians) required to aim directly at the target.
- w : The wind speed acting on the x-axis (positive = tailwind, negative = headwind).

6.3 Action Space

The agent operates in a continuous action space with two dimensions, scaled to $[-1, 1]$ for compatibility with the PPO algorithm. These are mapped to physical quantities as follows:

1. **Angle** (α): Mapped from $[-1, 1]$ to $[0^\circ, 85^\circ]$.
2. **Power** (P): Mapped from $[0, 1]$ to $[0, v_{max}]$, where $v_{max} = 35 \text{ m/s}$.

6.4 Reward Function

To overcome the sparsity of rewards in shooting tasks, a dense reward function was implemented based on the minimum distance the arrow achieves relative to the target during its flight. The reward R is calculated as:

$$R = \begin{cases} 100 - d_{min} & \text{if hit (collision occurs)} \\ -d_{min} & \text{if miss} \end{cases}$$

where d_{min} is the closest Euclidean distance the arrow reached to the target center throughout its entire trajectory. This encourages the agent to minimize error even when the target is missed, providing a smooth gradient for optimization.

6.5 Episode Termination

Since the environment models a single shot, every episode has a horizon of $T = 1$. The environment resets immediately after the arrow either hits the target, hits the ground ($y < 0$), or leaves the actionable boundary ($x > 50$).

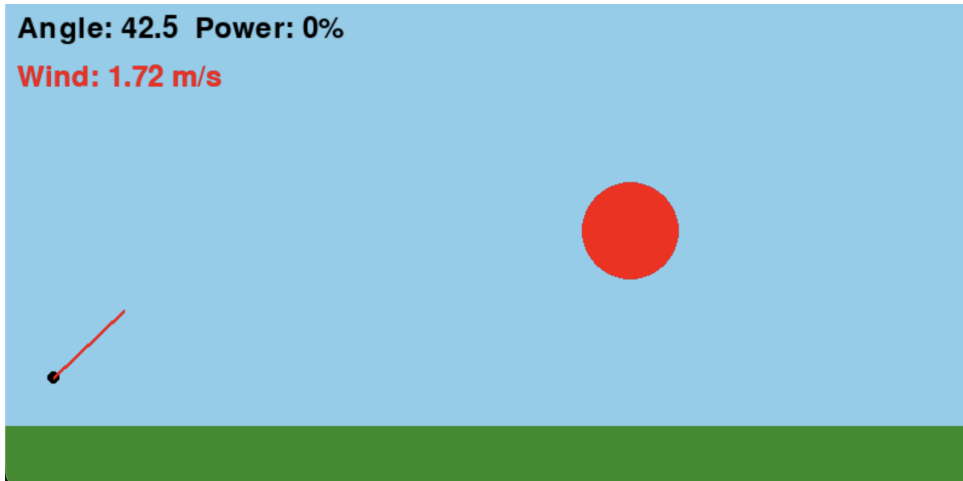


Figure 1: Archery Environment

References

- [1] Costa Huang. Ppo implementation details. <https://github.com/vwxyzjn/ppo-implementation-details/blob/main/ppo.py>, 2024. Accessed: 2026-02-09.